Scikit-learn Study Notes

Machine Learning Library for Python — Build ML Models in Minutes

For 10th Standard Students & ML Beginners

1. What is Scikit-learn?

Scikit-learn (sklearn) is the most popular Python library for Machine Learning. It gives you ready-made ML algorithms so you don't have to code them from scratch. You just give it data and ask it to learn patterns.

> Think of it like: A recipe book for ML. Each algorithm is a recipe. You give it ingredients (data), follow the steps (fit → predict), and get results (predictions).

What you can do with Scikit-learn:

- Classification — Is this email spam or not? Is this tumor malignant?
- Regression — What will the house price be? What will the temperature be tomorrow?
- Clustering — Group customers by shopping behavior (no labels needed)
- Dimensionality Reduction — Reduce 100 features to 10 important ones
- Model Evaluation — Check how good your model is
- Preprocessing — Scale, encode, and transform data for models

```
pip install scikit-learn
import sklearn
print(sklearn.__version__)   # check version
```

2. The Universal ML Pattern in Sklearn

Every single sklearn model follows the EXACT same 4 steps. Learn these once, use forever:

Step	Code	What Happens
1. Import model	`from sklearn.tree import DecisionTreeClassifier`	Load the algorithm
2. Create model	`model = DecisionTreeClassifier()`	Set up the model
3. Train model	`model.fit(X_train, y_train)`	Model learns from data
4. Predict	`predictions = model.predict(X_test)`	Model guesses on new data

```
# Complete minimal example:
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

```
X = [[5.1,3.5],[4.9,3.0],[6.7,3.1],[5.8,2.7]]   # features
y = [0, 0, 1, 1]                                  # labels
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
print(model.predict(X_test))
```

3. Data Preprocessing — Prepare Before Training

ML models are like picky eaters — they need data in the RIGHT format. Preprocessing fixes this.

3.1 Train-Test Split

Always split data: train the model on one part, test it on the other. This checks if the model truly learned or just memorized.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% for testing, 80% for training
    random_state=42,    # same split every time you run
    stratify=y          # keep class proportions equal
)
```

3.2 Feature Scaling

Many models (SVM, KNN, Neural Networks) get confused if features are on different scales (age: 0-100, salary: 0-100000). Scaling fixes this.

Scaler	Code	When to Use	Formula
StandardScaler	`StandardScaler()`	Most cases (mean=0, std=1)	(x - mean) / std
MinMaxScaler	`MinMaxScaler()`	When you need [0,1] range	(x - min) / (max - min)
RobustScaler	`RobustScaler()`	Data has many outliers	Uses median & IQR
Normalizer	`Normalizer()`	Each ROW sums to 1	x / ‖x‖

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # learn stats + transform
X_test_scaled  = scaler.transform(X_test)       # ONLY transform (don't refit!)
```

> ⚠ NEVER fit the scaler on test data! Only fit on training data, then transform test. Otherwise you 'leak' info about the test set.

3.3 Encoding Categorical Data

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
```

```
# Label Encoding — for target variable (y)
le = LabelEncoder()
y_encoded = le.fit_transform(['cat', 'dog', 'cat', 'bird'])
# Result: [1, 2, 1, 0]
```

```
# One-Hot Encoding — for categorical input features
ohe = OneHotEncoder(sparse_output=False)
X_encoded = ohe.fit_transform([['cat'], ['dog'], ['bird']])
```

3.4 Handle Missing Values

```
from sklearn.impute import SimpleImputer, KNNImputer
```

```
# Fill with mean/median/most_frequent
imputer = SimpleImputer(strategy='median')   # or 'mean', 'most_frequent'
X_filled = imputer.fit_transform(X)
```

```
# KNN-based imputation (smarter)
knn_imp = KNNImputer(n_neighbors=5)
X_filled = knn_imp.fit_transform(X)
```

3.5 Polynomial Features

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)  # adds x², xy, y² etc.
```

3.6 Pipeline — Chain Everything Together

Pipeline combines preprocessing + model in one object. Clean and mistake-proof.

```
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

```
pipe.fit(X_train, y_train)
pipe.predict(X_test)   # automatically preprocesses then predicts!
```

4. Classification Algorithms

Classification = predicting a CATEGORY (spam/not spam, cat/dog, 0/1/2).

Algorithm	Best For	Simple Code
Logistic Regression	`Binary/multi-class, baseline model`	LogisticRegression()
Decision Tree	`Easy to explain, works without scaling`	DecisionTreeClassifier()
Random Forest	`High accuracy, handles noise well`	RandomForestClassifier()
SVM	`High-dimensional data, text classification`	SVC()
KNN	`Small datasets, simple problems`	KNeighborsClassifier(k=5)
Naive Bayes	`Text classification, email spam`	GaussianNB()
Gradient Boosting	`Competitions, tabular data`	GradientBoostingClassifier()
Extra Trees	`Faster version of Random Forest`	ExtraTreesClassifier()

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=1.0, max_iter=1000, solver='lbfgs')
# C = regularization (smaller = more regularization)
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=5, min_samples_split=10)
# max_depth prevents overfitting
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
# n_estimators = number of trees
```

SVM (Support Vector Machine)

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
```

```
# kernel: 'linear', 'rbf', 'poly'
```

KNN (K-Nearest Neighbors)

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
```

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)
```

```
# Get class probabilities instead of hard predictions
model.predict_proba(X_test)   # returns [P(class0), P(class1)]
```

5. Regression Algorithms

Regression = predicting a NUMBER (house price, temperature, score).

Algorithm	Best For	Simple Code
Linear Regression	Simple relationships, baseline	LinearRegression()
Ridge Regression	Many features, prevent overfitting	Ridge(alpha=1.0)
Lasso Regression	Feature selection (makes some coef=0)	Lasso(alpha=1.0)
ElasticNet	Combines Ridge + Lasso	ElasticNet()
Decision Tree	Non-linear relationships	DecisionTreeRegressor()
Random Forest	Best general-purpose regressor	RandomForestRegressor()
SVR	Small dataset, non-linear	SVR(kernel='rbf')
Gradient Boosting	Best accuracy on tabular data	GradientBoostingRegressor()

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
```

```
# Basic linear regression
lr = LinearRegression()
lr.fit(X_train, y_train)
print('Coefficients:', lr.coef_)
print('Intercept:', lr.intercept_)
```

6. Clustering — Unsupervised Learning

Clustering = grouping similar data points WITHOUT labels. Used for customer segmentation, anomaly detection, data exploration.

Algorithm	Best For	Code
K-Means	Round-shaped clusters, known number of clusters	KMeans(n_clusters=3)
DBSCAN	Arbitrary shapes, finds outliers automatically	DBSCAN(eps=0.5, min_samples=5)
Agglomerative	Hierarchical grouping, any number of clusters	AgglomerativeClustering(n_clusters=3)
MeanShift	Doesn't need number of clusters specified	MeanShift()

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
labels = kmeans.labels_            # which cluster each point belongs to
centers = kmeans.cluster_centers_  # center of each cluster
```

```
# Find best number of clusters using Elbow method
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)   # lower = better (but overfitting risk)
```

7. Dimensionality Reduction

When you have too many features (100s of columns), use these to reduce to the most important ones.

7.1 PCA — Principal Component Analysis

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)          # reduce to 2 dimensions
X_reduced = pca.fit_transform(X)
print(pca.explained_variance_ratio_)  # how much info each component holds
```

```
# Keep 95% of information automatically
pca = PCA(n_components=0.95)
```

7.2 Feature Selection

```
from sklearn.feature_selection import SelectKBest, f_classif, chi2
from sklearn.feature_selection import RFE
```

```
# Select top 10 best features
selector = SelectKBest(score_func=f_classif, k=10)
X_new = selector.fit_transform(X, y)
```

```
# Recursive Feature Elimination
rfe = RFE(estimator=RandomForestClassifier(), n_features_to_select=10)
X_new = rfe.fit_transform(X, y)
```

8. Model Evaluation — How Good is Your Model?

Training a model is only half the job. You must measure how well it works on NEW data.

8.1 Classification Metrics

Metric	Code	What It Means
Accuracy	`accuracy_score(y_test, y_pred)`	% of correct predictions
Precision	`precision_score(y_test, y_pred)`	Of predicted positives, how many are actually positive?
Recall	`recall_score(y_test, y_pred)`	Of actual positives, how many did we catch?
F1 Score	`f1_score(y_test, y_pred)`	Balance between precision & recall
ROC AUC	`roc_auc_score(y_test, y_prob)`	Area under ROC curve (1=perfect)
Confusion Matrix	`confusion_matrix(y_test, y_pred)`	Table of TP, TN, FP, FN
Full Report	`classification_report(y_test, y_pred)`	All metrics in one table

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, roc_auc_score)
```

```
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))  # shows all metrics
```

8.2 Regression Metrics

Metric	Code	What It Means
MSE	`mean_squared_error(y_test, y_pred)`	Average squared error (penalizes big errors more)

RMSE	`mean_squared_error(..., squared=False)`	Square root of MSE (in original units)
MAE	`mean_absolute_error(y_test, y_pred)`	Average absolute error (easy to understand)
R^2 Score	`r2_score(y_test, y_pred)`	1 = perfect, 0 = just guessing the mean

8.3 Cross-Validation — Better Than Single Split

Cross-validation splits data into multiple folds and tests on each. Much more reliable than single train-test split.

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold
```

```
# 5-fold cross validation
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'Mean: {scores.mean():.3f} ± {scores.std():.3f}')
```

```
# Stratified KFold (keeps class balance in each fold)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='f1')
```

9. Hyperparameter Tuning — Making Models Better

Hyperparameters are settings you choose before training (like max_depth=5). Finding the best settings is called tuning.

9.1 Grid Search — Try All Combinations

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10]
}
```

```
grid_search = GridSearchCV(
    RandomForestClassifier(),
    param_grid,
    cv=5,               # 5-fold cross validation
    scoring='accuracy',
    n_jobs=-1,          # use all CPU cores
    verbose=1           # show progress
)
grid_search.fit(X_train, y_train)
print('Best Params:', grid_search.best_params_)
```

```
print('Best Score:', grid_search.best_score_)
best_model = grid_search.best_estimator_
```

9.2 Random Search — Faster Than Grid Search

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
```

```
param_dist = {
    'n_estimators': randint(50, 500),
    'max_depth': randint(3, 20)
}
rand_search = RandomizedSearchCV(model, param_dist, n_iter=50, cv=5)
rand_search.fit(X_train, y_train)
```

10. Ensemble Methods — Combine Multiple Models

Instead of one model, combine many. Like asking 100 people and taking a vote — more accurate!

Method	How it Works	Code
Bagging	`Train many models on different samples, take vote`	BaggingClassifier()
Random Forest	`Many decision trees on random features`	RandomForestClassifier()
AdaBoost	`Each tree fixes errors of previous`	AdaBoostClassifier()
Gradient Boosting	`Builds trees step by step to reduce error`	GradientBoostingClassifier()
Voting Classifier	`Take vote of multiple different models`	VotingClassifier()
Stacking	`Use model outputs as input to another model`	StackingClassifier()

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

```
# Combine 3 different models
voting = VotingClassifier([
    ('lr', LogisticRegression()),
    ('dt', DecisionTreeClassifier()),
    ('svm', SVC(probability=True))
], voting='soft')    # 'soft' uses probabilities
voting.fit(X_train, y_train)
```

11. Saving & Loading Models

```
import joblib
```

```
# Save model
joblib.dump(model, 'my_model.pkl')
joblib.dump(scaler, 'scaler.pkl')   # save scaler too!
```

```
# Load model (later, or in another script)
model = joblib.load('my_model.pkl')
scaler = joblib.load('scaler.pkl')
```

```
# Use loaded model
new_data = scaler.transform(new_X)  # scale first!
prediction = model.predict(new_data)
```

12. Complete ML Workflow — Put It All Together

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.pipeline import Pipeline
import joblib
```

```
# STEP 1: Load data
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']
```

```
# STEP 2: Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

```
# STEP 3: Build pipeline
pipe = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100))
])
```

```
# STEP 4: Train
pipe.fit(X_train, y_train)
```

```
# STEP 5: Evaluate
y_pred = pipe.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
# STEP 6: Cross-validate
cv_scores = cross_val_score(pipe, X, y, cv=5)
print(f'CV Score: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}')
```

```
# STEP 7: Save
joblib.dump(pipe, 'final_model.pkl')
```

13. Algorithm Selection Guide

Your Problem	Try First	If Need Better
Binary classification	`Logistic Regression`	Random Forest, XGBoost
Multi-class classification	`Random Forest`	Gradient Boosting
Text classification	`Naive Bayes, Logistic Reg`	SVM with TF-IDF
Small dataset (<1000)	`SVM, KNN`	Logistic Regression
Large dataset (>100K)	`SGDClassifier, Linear SVM`	Random Forest
Predict a number	`Linear Regression`	Random Forest Regressor
Group similar items	`K-Means`	DBSCAN (for odd shapes)
Reduce features	`PCA`	LDA (if you have labels)

14. Quick Reference — Most Used Classes

Category	Class	Purpose
Split	`train_test_split()`	Split data into train/test
Split	`cross_val_score()`	K-fold cross validation
Preprocess	`StandardScaler()`	Mean=0, Std=1 scaling
Preprocess	`MinMaxScaler()`	Scale to [0,1]
Preprocess	`SimpleImputer()`	Fill missing values
Preprocess	`LabelEncoder()`	Encode labels to numbers
Preprocess	`OneHotEncoder()`	One-hot encode categories
Pipeline	`Pipeline([])`	Chain steps together
Classifier	`LogisticRegression()`	Baseline classifier
Classifier	`RandomForestClassifier()`	Powerful tree ensemble
Classifier	`SVC()`	Support Vector Machine
Regressor	`LinearRegression()`	Baseline regressor
Regressor	`RandomForestRegressor()`	Powerful tree ensemble
Cluster	`KMeans()`	Group into k clusters
Reduce	`PCA()`	Reduce dimensions
Tune	`GridSearchCV()`	Find best parameters
Metrics	`accuracy_score()`	Classification accuracy
Metrics	`r2_score()`	Regression quality
Metrics	`classification_report()`	Full classification metrics
Save	`joblib.dump()`	Save trained model

🎓 **You now know Scikit-learn! Build your first ML project today and keep practicing.**